

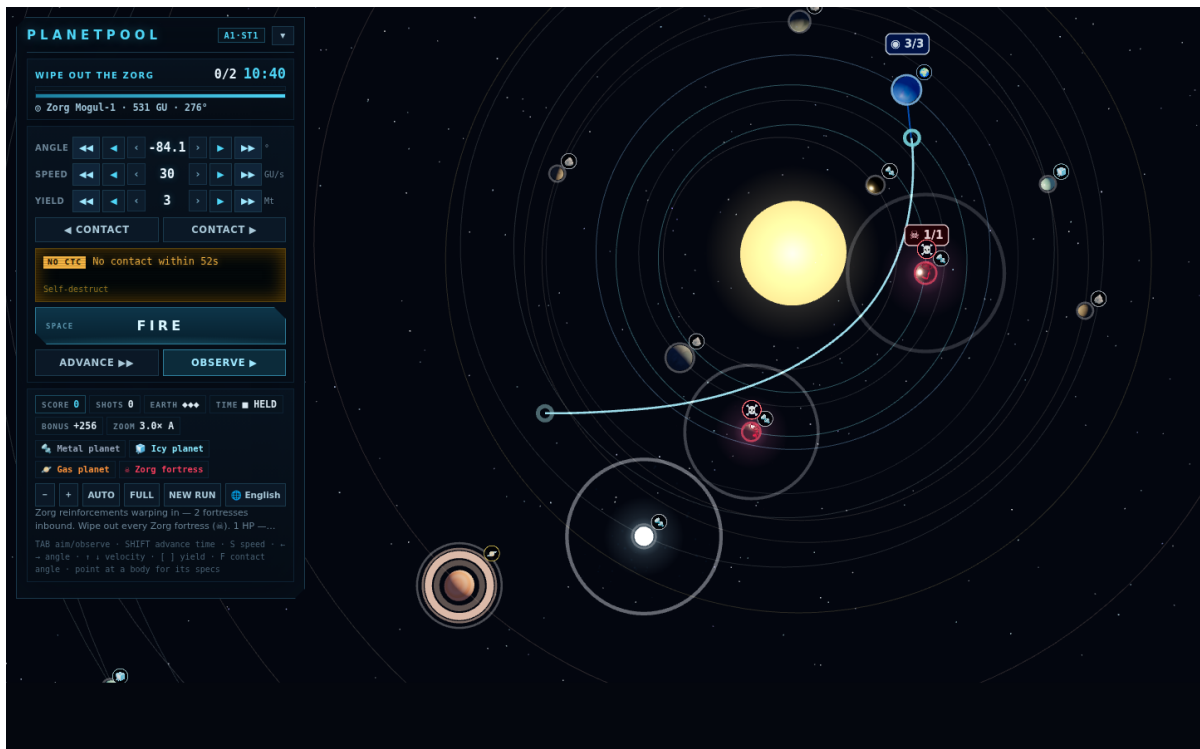
PLANETPOOL × Codex

행성당구 로그라이크 — AI와 함께 만든 과정 문서

세 줄 요약

- ① 핵미사일을 중력 스윙바이로 휘어 쏘는 당구 — 화면의 모든 행성이 실제 N체 중력 시뮬레이션으로 움직입니다.
- ② 궤도역학·수치 적분 같은 전문 지식은 Codex가 채우고, 재미의 판정과 모든 방향 결정은 사람이 했습니다.
- ③ 이미지·오디오 등 외부 에셋 파일 0개 — 그래픽은 three.js로, 효과음까지 전부 코드로 절차 생성합니다.

PLANETPOOL은 핵미사일을 행성의 중력에 태워 휘어 쏘는 우주 당구 로그라이크입니다. 행성들은 고정된 장애물이 아니라 뉴턴 중력으로 서로를 끌어당기는 N체 시뮬레이션 위에서 살아 움직이고, 직선으로 쏘면 실패하며 궤적을 휘어야 이깁니다 — 궤도역학이라는 전문 지식 자체가 게임의 정체성인 프로젝트입니다.



실제 플레이 화면 — 예측선(하늘색)이 태양 중력에 붙잡혀 휘어 들어간다. 직선 조준으로는 절대 닿지 않는 판. 왼쪽은 LCD풍 사격통제 패널(각도·속도·탄두 위력·교신 로그).

기술 스택	바닐라 JavaScript(ES 모듈) + three.js · ZzFX 절차 생성 효과음 · 빌드 도구 없음, 정적 호스팅만으로 배포
물리	뉴턴 중력 + 소프트닝, KDK 리프로그(심플렉틱) 적분, 스윙바이 판정, 임펄스/파쇄 모델
제작 방식	사람의 방향 제시 → Codex 구현 → 직접 플레이 검증의 짧은 이터레이션 반복 · 5개 언어 지원(한/영/일/중/간체·번체)

직접 확인하기

저장소: github.com/ktrack723/OpenAIGame2026 — 클론 후 `npm run serve` → `localhost:4173` (브라우저만 있으면 실행, 설치할 의존성 없음). 빠른 반복 플레이는 `?nointro=1`, 같은 판 재현은 `?seed=숫자`.

1. Codex를 어디에 사용했나요?

개발 전 과정에서 역할을 나눠 사용했습니다. 사람이 방향을 정하고, Codex가 지식과 구현을 채웠습니다.

영역	Codex가 한 일
기술 선정	활용 라이브러리와 배포 방식 탐색·비교·선정 (three.js 로컬 번들 + 무빌드 정적 배포 결정 근거 제공)
도메인 지식	중력계수($\mu=GM$), 탈출속도, 행성 질량, 공전속도, N체 문제의 수치 시뮬레이션 방법 등 천문학 지식을 검색·취합해 게임 수식으로 정리
아키텍처	주어진 제약(무빌드·바닐라 JS) 안에서 이상적인 프로그래밍 패턴과 코드 아키텍처를 설계하고 코드 작성
오디오 구현	주요 액션별 효과음(발사·레이저·폭발·교신 치지직)을 오디오 파일 없이 ZzFX 합성 파라미터로 설계·구현
UI 폴리싱	이터레이션을 직접 실행하며 UI 문구·배치·색·크기 반복 조정
시퀀스 구현	인트로·튜토리얼처럼 깊은 로직 없이 손이 많이 가는 연출 시퀀스를 대량 작성
리팩토링	폴더 구조 정리와 코드 리팩토링 (core / game / render / ui / i18n 분리)
다국어	전체 텍스트 번역 및 5개 언어 대응 체계 구축

2. 어떤 기능을 구현했나요?

- N체 물리 엔진 — 태양 + 행성 상호 중력(소프트닝 포함)을 매 프레임 계산하고, 장시간 궤도가 무너지지 않도록 에너지를 보존하는 KDK 리프프로그 적분기로 진행
- 스윙바이 게임플레이 — 조준·발사 시스템, 수천 스텝을 미리 도는 궤적 예측선, 근접 통과·굴절각 기반 스윙바이 판정과 점수 배율
- 로그라이크 진행 — 판마다 다시 차오르는 성계, 카이퍼 벨트, 워프 증원, 조르그 레이저·모함 등 라운드 단위 위협의 성장
- 액션별 SFX — 발사음, 레이저, 폭발, 교신 치지직 노이즈까지 주요 액션 하나하나를 ZzFX로 실시간 합성 (사전 렌더 캐시 + 마스터 컴프레서)
- 연출과 온보딩 — 오프닝 컷신, 단계별 튜토리얼, 3D 발사기 연출, 필요한 설명을 필요한 판에만 보여주는 안내 시스템
- UI/UX — LCD풍 사격통제 HUD, 궤도 표시, 관측 배속, 언어 선택을 포함한 5개 언어 인터페이스

3. 어떤 문제를 해결했나요?

- 궤도가 터지는 문제 — 일반적인 RK4 적분은 에너지 드리프트로 몇 분 뒤 궤도가 새어나갑니다. Codex가 천체 시뮬레이션 표준인 심플렉틱(리프프로그) 적분을 제안·구현해 장시간 안정성을 확보했습니다.
- 예측선 프레임 드랍 — 예측선 한 번에 수천 스텝 × 천체 수만큼 객체가 생성되어 행성이 20개로 늘자 프레임이 떨어졌습니다. 가속도 버퍼 재사용으로 할당을 제거했습니다.
- 웹 오디오의 함정들 — 브라우저 자동재생 정책은 첫 사용자 제스처(CLICK TO START)에서 AudioContext를 깨워 통과하고, 음량 급변의 클릭 노이즈는 최소 5ms 램프로, 같은 효과음 폭주는 30ms 중복 억제 + 마스터 컴프레서로 눌렀습니다. 효과음은 사전 렌더 후 캐시해 발사 순간의 프레임 드랍을 막았습니다.
- 네트워크 없이도 도는 배포 — CDN이 막힌 환경에서도 로딩이 멈추지 않도록 라이브러리를 로컬 번들로 내장하고, 로딩 실패를 화면에 그대로 알려주는 부트 에러 처리를 붙였습니다.
- "당구인데 볼링이 되는" 문제 — 직선으로 쏘면 이기는 판이 나오지 않도록, 기획서의 차폐도·당구감 계수 같은 규칙을 수식으로 만들어 목표 경험을 코드로 강제했습니다.

4. 사람이 직접 결정한 부분은 무엇인가요?

- 게임의 정체성 — "직사는 실패다, 궤적이 휘어야 한다, 판은 살아 있다"라는 목표 경험의 정의와 우선순위
- AI에게 일을 시키는 방식 자체의 설계 — 기획서의 수식 20개를 "매 프레임 도는 것 / 판을 만들 때 한 번만 도는 것 / 코드에는 안 들어가고 밸런스 튜닝에만 쓰는 것" 세 갈래로 사람이 직접 분류해, AI가 "무엇을 구현하고 무엇을 무시해도 되는지" 판단 없이 받아먹을 수 있는 스펙으로 만들었습니다
- 재미의 판정 — 매 이터레이션을 직접 플레이하며 밸런스(체력, 대응시간, 배속, 안테 구성)를 감각으로 조정하고 다음 방향을 지시
- 취사선택 — Codex가 정리한 수식·설계안 중 무엇을 채택하고 무엇을 버릴지(예: 과한 천문학적 정밀도는 화면에서 구분 불가하므로 단순화) 최종 결정

5. 만약 AI가 없었다면 — 이 게임에서 AI를 쓴 의미

행성당구는 아이디어만으로는 만들 수 없는 장르입니다. 게임의 재미 자체가 진짜 궤도역학 위에 서 있기 때문입니다. 전공자가 아닌 개인이 이 게임을 만들려면 다음 지식을 스스로 학습해야 합니다.

- 중력계수 $\mu=GM$ 과 질량-반지름-내구도의 파생 관계, 탈출속도와 공전속도의 계산
- N체 문제는 해석적 해가 없다는 사실과, 수치 적분기 선택(왜 RK4가 아니라 심플렉틱인가)
- 스윙바이(중력 어시스트)의 원리와 굴절각, 안정 궤도 배치를 위한 힐 반경 같은 개념

이것은 교양서 몇 권이 아니라 항공우주역학 교재와 수치해석 자료를 오가며 몇 주에서 몇 달을 들여야 하는 학습량이고, 지식을 얻어도 그것을 60fps로 도는 게임 코드로 옮기는 것은 또 다른 벽입니다. Codex는 이 두 벽을 동시에 낮췄습니다 — 필요한 전문 지식을 그 자리에서 검색·취합해 주고, 그 지식을 곧바로 검증 가능한 코드로 만들어 주었습니다. 사람은 "무엇이 재미있는가"에만 집중하고, AI가 "어떻게 물리적으로 옮겨 만드는가"를 채운 분업 — 그 결과 일반 개발자 혼자서는 시작조차 어려웠을 실제 N체 시뮬레이션 게임이 완성되었습니다.

"AI는 코드를 대신 쳐 준 것이 아니라, 이 게임이 존재할 수 있는 지식의 문턱을 없애 주었다."